

A PBL-II (AIM2270) SUMMARY REPORT on

SpamGuard-NLP

Submitted to Manipal University Jaipur

Towards the partial fulfillment for the Award of the Degree of

B. Tech Computer Science and Engineering (Artificial Intelligence and Machine Learning)

By

Granth Joshi

2427010466



**MANIPAL UNIVERSITY
JAIPUR**

Under the guidance of

Dr. Hemlata Parmar

Department of Artificial Intelligence and Machine Learning

Manipal University Jaipur

Jaipur, Rajasthan

CERTIFICATE

This is to certify that the project entitled SpamGuard-NLP is a Bonafide work carried out as part of the course AIM2270, under my guidance from Jan 2026 to May 2026 by Granth Joshi (2427010466), student of B. Tech (hons.) Computer Science and Engineering (AIML), 4th Semester at the Department of Artificial Intelligence and Machine Learning, Manipal University Jaipur, during the academic semester 4th in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AIML), at MUJ, Jaipur.

Dr. Hemlata Parmar

*Project Guide, Dept of AIML
Manipal University Jaipur*

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to **Prof. (Dr.) Kuldip Singh Sangwan**, Dean, Faculty of Science, Technology and Architecture, for his constant encouragement, valuable guidance, and support throughout the PBL-2 course.

We are also deeply thankful to **Dr. Roheet Bhatnagar**, Professor & Associate Dean, Department of Computer Science & Engineering, for his insightful suggestions, academic guidance, and continuous motivation, which greatly contributed to the successful completion of this project.

Our heartfelt thanks go to **Dr. Deepak Panwar**, Associate Professor & Head of the Department, Department of Artificial Intelligence & Machine Learning, for providing the necessary facilities, support, and a conducive learning environment that enabled us to carry out this project effectively.

We would like to extend our **special thanks to the PBL Coordinators** for their continuous guidance, coordination, and timely support throughout the duration of the project.

We are especially grateful to **our project guide** for their invaluable mentorship, constructive feedback, constant encouragement, and dedicated support at every stage of the project, which played a crucial role in its successful completion.

We also thank all faculty members and staff involved in PBL-2 for their cooperation and support.

ABSTRACT

The rapid growth of mobile communication has led to a significant increase in SMS-based spam, including phishing attacks, fraudulent schemes, and malicious advertisements. These messages pose serious risks to users by compromising privacy and financial security. In multilingual regions like India, the problem becomes more complex due to the presence of multiple languages such as Hindi, which traditional English-based spam detection systems fail to handle effectively.

Existing spam detection systems primarily rely on single-language datasets and often lack the capability to process multilingual inputs. Moreover, many approaches are computationally expensive or not suitable for real-time deployment. This highlights the need for a lightweight, scalable, and multilingual spam detection system capable of delivering fast and accurate predictions.

To address these challenges, this project proposes SpamGuard, a multilingual SMS spam detection system that combines Natural Language Processing techniques with machine learning. The system adopts a translation-first approach, converting non-English messages into English, followed by text preprocessing, TF-IDF-based feature extraction, and classification using a Multinomial Naive Bayes model. The system is deployed using a Flask-based web application to enable real-time inference.

The experimental results demonstrate that the proposed system achieves an accuracy of 97.13%, with strong performance across both spam and ham classes. The system is efficient, easy to deploy, and suitable for real-world applications. Overall, SpamGuard provides a practical solution for multilingual spam detection while maintaining a balance between performance and computational efficiency.

**LIST OF
FIGURES**

Figure No	Figure Title	Page No
1	SpamGuard System Architecture	
2	Spam Detection Pipeline	
3	Classification	

Contents

		Page No
Acknowledgement		i
Abstract		ii
List Of Figures		iii
List Of Tables		vi
Chapter 1	INTRODUCTION	
1.1	Introduction to work done	
1.2	Project Objective	
1.3	Organization of Report	
Chapter 2	BACKGROUND MATERIAL	
2.1	Conceptual Overview	
2.2	Technologies Involved	
2.3	Literature Review	
.		
.		
.		
Chapter 3	METHODOLOGY	
3.1	Detailed methodology	
3.2	block diagrams	
2.4	Outcome of Literature Review	
3.3	Algorithmic Steps	
3.4	Design Considerations	
.		
.		
.		
Last Chapter	CONCLUSIONS	
4.1	Conclusions	
REFERENCES		

CHAPTER 1: INTRODUCTION

1.1 Introduction

Spam messages are unsolicited and often malicious SMS communications that are widely used for advertising, phishing attacks, and fraudulent activities. With the rapid growth of mobile communication and internet accessibility, the volume of SMS traffic has increased significantly, making spam detection an essential requirement. These spam messages not only degrade user experience but also expose users to serious risks such as financial fraud, identity theft, and data privacy violations.

In real-world scenarios, spam messages are designed to appear legitimate, often mimicking trusted sources such as banks, government agencies, or service providers. This makes manual identification difficult for users and increases the need for automated detection systems.

The challenge becomes more complex in multilingual environments like India, where users communicate in multiple languages such as Hindi, English, and regional languages. Most traditional spam detection systems are trained on English datasets and are not capable of effectively processing multilingual or mixed-language inputs. As a result, their performance drops significantly in real-world applications.

Furthermore, existing systems often focus on accuracy but ignore practical deployment aspects such as computational efficiency and real-time processing. This creates a gap between research-based models and real-world usable systems.

Therefore, there is a strong need for a spam detection system that is not only accurate but also capable of handling multilingual input efficiently. The motivation behind this project is to develop a lightweight, scalable, and real-time system that can effectively classify SMS messages across different languages.

1.2 Project Objectives

The main objectives of the SpamGuard system are:

- To design and develop a multilingual SMS spam detection system

- To handle code-mixed and non-English text inputs
- To implement Natural Language Processing (NLP) techniques for text preprocessing
- To apply TF-IDF feature extraction for representing textual data numerically
- To use a Multinomial Naive Bayes classifier for efficient spam classification
- To build a real-time web-based system using Flask
- To achieve high accuracy while maintaining low computational complexity
- To create a system that is scalable and easy to deploy

1.3 Organization of Report

This report is organized into multiple chapters. Chapter 1 introduces the problem and objectives of the project. Chapter 2 discusses the background concepts and literature review. Chapter 3 explains the methodology and workflow of the system. Chapter 4 describes the system design and architecture. Chapter 5 presents the implementation details and experimental results. Finally, Chapter 6 concludes the work and suggests future improvements.

CHAPTER 2: BACKGROUND

2.1 Conceptual Overview

Spam detection is a classification problem within the domain of Natural Language Processing (NLP), where the goal is to categorize text messages into predefined classes such as spam and ham (non-spam). NLP techniques enable machines to understand, interpret, and process human language, making them essential for analyzing textual data like SMS messages.

Several machine learning algorithms have been applied to spam detection, including Naive Bayes, Support Vector Machines (SVM), Decision Trees, and more recently, deep learning models such as Recurrent Neural Networks (RNNs) and Transformers. Among these, Naive Bayes remains a popular choice due to its simplicity, efficiency, and strong performance on text classification tasks.

TF-IDF (Term Frequency–Inverse Document Frequency) is a statistical method used to evaluate the importance of a word in a document relative to a collection of documents. It helps in identifying significant words that contribute to classification, while reducing the impact of commonly occurring words.

In addition to feature extraction, preprocessing plays a critical role in improving model performance. Techniques such as tokenization, stopword removal, and normalization help in cleaning the data and reducing noise.

The combination of TF-IDF and Multinomial Naive Bayes provides a strong baseline model for spam detection, especially when dealing with short text data like SMS messages.

2.2 Technologies Involved

The following technologies are used in the development of the SpamGuard system:

- Python – Used as the primary programming language due to its simplicity and extensive support for machine learning libraries
- Flask – A lightweight web framework used to build the backend API and enable real-time interaction

- Scikit-learn – Provides tools for TF-IDF vectorization and implementation of the Naive Bayes classifier
- HTML, CSS, JavaScript – Used for designing the frontend interface for user interaction
- Translation APIs – Used to convert multilingual input into English for uniform processing

2.3 Literature Review

Previous research in spam detection has explored a variety of techniques ranging from traditional machine learning algorithms to advanced deep learning models. Naive Bayes and Support Vector Machines have been widely used due to their effectiveness in handling text classification problems. More recent approaches involve deep learning models that can capture semantic relationships in text.

However, most of these studies are limited to English-language datasets and do not address the challenges of multilingual or code-mixed text. Additionally, deep learning models often require large datasets and high computational resources, making them less suitable for real-time applications.

2.4 Outcome of Literature Review

The literature review reveals that while significant progress has been made in spam detection, existing systems lack the ability to effectively handle multilingual and code-mixed data. There is also a trade-off between model complexity and real-time performance.

This highlights the need for a system that balances accuracy, efficiency, and scalability. The proposed SpamGuard system addresses this gap by using a translation-based approach combined with lightweight machine learning techniques to achieve real-time multilingual spam detection.

CHAPTER 3: METHODOLOGY

3.1 Detailed Methodology

The methodology of the SpamGuard system is designed to efficiently classify SMS messages as spam or ham by integrating Natural Language Processing (NLP) techniques with machine learning. The system follows a structured pipeline that ensures accurate, scalable, and real-time prediction while supporting multilingual inputs.

The overall workflow consists of multiple stages, including data acquisition, preprocessing, feature extraction, model training, and prediction. Each stage plays a critical role in improving the performance of the system.

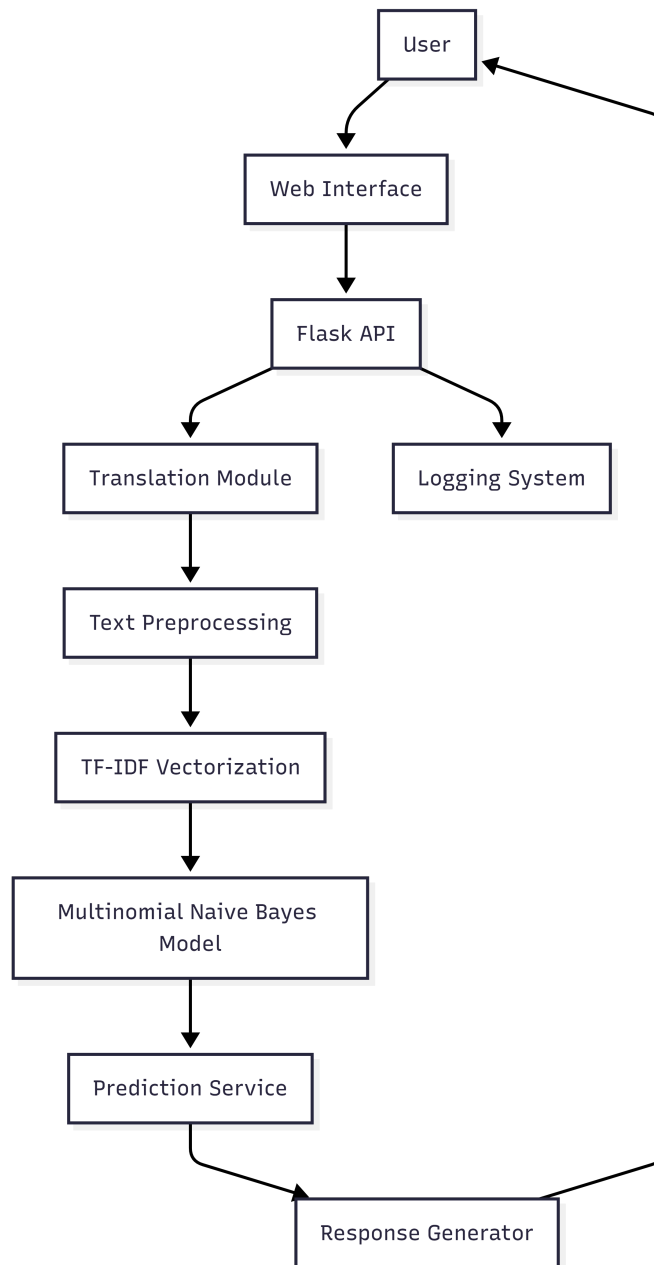
3.1.1 Data Collection and Dataset Description

The dataset used in this project consists of labeled SMS messages categorized into two classes: spam and ham (non-spam). The dataset contains a diverse set of messages with varying lengths, vocabulary, and writing styles. Spam messages typically include promotional content, urgent offers, and suspicious links, whereas ham messages represent normal communication.

The dataset is stored in a structured format (CSV file) with two columns:

- **Label:** Indicates whether the message is spam or ham
- **Message:** Contains the SMS text

To ensure effective learning, the dataset includes a mix of short and long messages, as well as messages containing special characters, abbreviations, and informal language patterns.



3.1.2 Multilingual Handling

One of the key challenges addressed in this project is the handling of multilingual and code-mixed text. Since the model is trained primarily on English data, it is essential to standardize input from different languages.

To achieve this, the system adopts a translation-first approach, where non-English messages are converted into English before further processing. This allows the system to maintain a single unified model instead of training separate models for each language.

Although translation may introduce minor inaccuracies, important spam-related keywords such as “offer,” “win,” and “urgent” are generally preserved, ensuring reliable classification performance.

3.1.3 Text Preprocessing

Raw SMS data often contains noise such as punctuation, special symbols, inconsistent capitalization, and irrelevant words. Preprocessing is a crucial step to clean and normalize the text before feature extraction.

The following preprocessing steps are applied:

- Lowercasing: All text is converted to lowercase to ensure uniformity
- Removal of Punctuation and Special Characters: Eliminates unnecessary symbols that do not contribute to classification
- Tokenization: Splits text into individual words or tokens
- Stopword Removal: Removes commonly used words such as “the,” “is,” and “and” that do not carry meaningful information
- Normalization: Ensures consistency in word representation

These steps help reduce noise and improve the quality of input data, leading to better model performance.

3.1.4 Feature Extraction using TF-IDF

Since machine learning models require numerical input, the textual data must be converted into a suitable numerical format. This is achieved using the Term Frequency–Inverse Document Frequency (TF-IDF) technique.

TF-IDF assigns a weight to each word based on:

- Its frequency in the current message (Term Frequency)
- Its rarity across all messages (Inverse Document Frequency)

Words that appear frequently in spam messages but rarely in normal messages receive higher weights. For example, words like “free,” “winner,” and “urgent” are strong indicators of spam and are assigned higher importance.

The output of this stage is a feature vector representing each message in numerical form, which is then used for classification.

3.1.5 Classification using Multinomial Naive Bayes

The classification task is performed using the Multinomial Naive Bayes (MNB) algorithm, which is widely used for text classification problems.

The model works by calculating the probability of a message belonging to a particular class (spam or ham) based on the frequency of words in the message. It assumes that the features (words) are conditionally independent, which simplifies computation and improves efficiency.

The reasons for choosing Multinomial Naive Bayes include:

- High performance on text data
- Fast training and prediction
- Low computational cost
- Suitable for sparse datasets

Despite its simplicity, the model provides strong baseline performance and is well-suited for real-time applications.

```

... Accuracy: 0.9713004484304932

Classification Report:
              precision    recall  f1-score   support

     ham         0.97         1.00         0.98         956
     spam         1.00         0.80         0.89         159

 accuracy          0.97         0.97         0.97         1115
  macro avg         0.98         0.90         0.94         1115
weighted avg         0.97         0.97         0.97         1115

Confusion Matrix:
[[956   0]
 [ 32 127]]

```

3.1.6 Model Training and Evaluation

The dataset is divided into training and testing sets to evaluate model performance. The training set is used to train the classifier, while the testing set is used to measure accuracy and generalization.

The performance of the model is evaluated using the following metrics:

- Accuracy: Overall correctness of predictions
- Precision: Ability to correctly identify spam messages
- Recall: Ability to detect all actual spam messages
- F1-Score: Balance between precision and recall

The results indicate that the model achieves high accuracy and performs well on both classes.

3.1.7 Real-Time Prediction using Flask

To make the system usable in real-world scenarios, a web-based interface is developed using Flask. The trained model is integrated into the backend, allowing users to input messages and receive predictions instantly.

The workflow is as follows:

1. User enters a message in the web interface
2. The message is sent to the Flask backend via an API request
3. The backend processes the message through the pipeline
4. The model predicts the class (spam or ham)
5. The result is returned to the user along with a confidence score

This setup enables real-time inference and makes the system interactive and user-friendly.

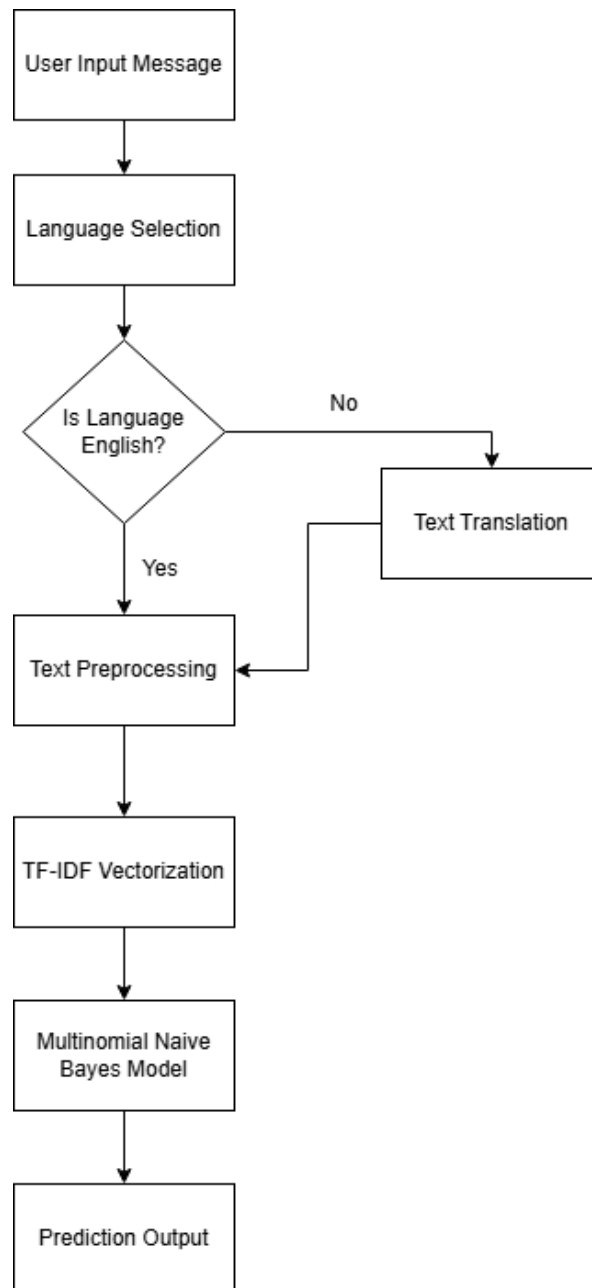
3.2 Block Diagram

The SpamGuard system follows a sequential pipeline that can be represented using a block diagram.

The workflow begins with user input, where the message is entered into the system. The input is then passed to the translation module, which converts multilingual text into English.

The translated text is processed by the preprocessing module, where it is cleaned and normalized. The cleaned text is then passed to the feature extraction module, which converts it into numerical vectors using TF-IDF.

These vectors are then fed into the classification model (Naive Bayes), which predicts whether the message is spam or ham. Finally, the output is displayed to the user along with a confidence score.



3.3 Algorithmic Steps

The overall algorithm for spam detection is as follows:

1. Input SMS message from user
2. Detect language and translate to English (if required)
3. Apply preprocessing techniques
4. Convert text into TF-IDF feature vectors
5. Load trained Naive Bayes model

6. Predict class label (spam or ham)
7. Output result with confidence score

3.4 Design Considerations

While designing the methodology, the following considerations were taken into account:

- Efficiency: The system should provide fast predictions
- Scalability: The model should handle large volumes of messages
- Simplicity: The design should be easy to implement and maintain
- Accuracy: The model should achieve high classification performance
- Flexibility: The system should support future improvements

CHAPTER 4: CONCLUSIONS

4.1 Conclusions

The SpamGuard project successfully demonstrates the design and implementation of a multilingual SMS spam detection system using Natural Language Processing and machine learning techniques. The system effectively addresses the limitations of traditional spam detection methods by incorporating support for multilingual and code-mixed text, which is highly relevant in real-world scenarios, especially in linguistically diverse regions like India.

The proposed approach utilizes a translation-first strategy, followed by text preprocessing, TF-IDF feature extraction, and classification using a Multinomial Naive Bayes model. This combination ensures a balance between computational efficiency and classification accuracy. The system is capable of handling noisy SMS data and identifying spam messages based on textual patterns and keyword importance.

The integration of a Flask-based web application enables real-time prediction, making the system practical and user-friendly. Users can input messages and receive instant classification results along with confidence scores, enhancing usability and accessibility.

The experimental results indicate that the model achieves an accuracy of 97.13%, with strong performance in terms of precision and recall. This demonstrates that even with a relatively simple machine learning approach, high accuracy can be achieved when combined with effective preprocessing and feature engineering techniques.

Overall, SpamGuard provides a lightweight, scalable, and efficient solution for multilingual spam detection. The modular architecture allows for future enhancements such as integration of deep learning models, improved translation mechanisms, and cloud-based deployment. The system thus serves as a strong foundation for further research and development in the field of intelligent spam filtering.

REFERENCES

Manning, C. D., Raghavan, P., & Schütze, H., "Introduction to Information Retrieval," Cambridge University Press, 2008.

Jurafsky, D., & Martin, J. H., "Speech and Language Processing," Pearson, 2019.

Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 2011.

McCallum, A., & Nigam, K., "A Comparison of Event Models for Naive Bayes Text Classification," *AAAI Workshop*, 1998.

Ramos, J., "Using TF-IDF to Determine Word Relevance in Document Queries," 2003.

Bird, S., Klein, E., & Loper, E., "Natural Language Processing with Python," O'Reilly Media, 2009.

Flask Documentation, "Flask Web Framework," Available:
<https://flask.palletsprojects.com/>
Google Translate API Documentation, Available:
<https://cloud.google.com/translate>

Zhang, H., "The Optimality of Naive Bayes," *Proceedings of FLAIRS*, 2004.

Sebastiani, F., "Machine Learning in Automated Text Categorization," *ACM Computing Surveys*, 2002.